GitHub

Complete Mastery Guide

From Absolute Beginner to Industry Level

12	60+	8	3
Modules	Git Commands	Interview Q&A	Projects

AUTHORS
B. Pavithra & K. Ranjana Devi
GUIDED BY:
Mr. S. Saravanan
Faculty, Women's Engineering College.

GITHUB
⌥ github.com/pavithraB-wec
⌥ github.com/ranjanadevi1802

ISE Student Edition · 2026 · Women's Engineering College, Puducherry

📋 TABLE OF CONTENTS

MODULE	TOPIC	Pg
Cover Page	Book Cover & Introduction	1
Index	Table of Contents	2
Module 1	Fundamentals — Git & GitHub	5
	▸ 1.1 What is Version Control?	
	▸ 1.2 What is Git?	
	▸ 1.3 What is GitHub?	
	▸ 1.4 Git vs GitHub	
	▸ 1.5 Core Vocabulary	
Module 2	Setup & Installation	8
	▸ 2.1 Install Git (Win/Mac/Linux)	
	▸ 2.2 First-Time Configuration	
	▸ 2.3 Connect to GitHub (HTTPS & SSH)	
Module 3	Basic Commands with Examples	10
	▸ 3.1 git init	
	▸ 3.2 git clone	
	▸ 3.3 git status	
	▸ 3.4 git add	
	▸ 3.5 git commit	
	▸ 3.6 git log	
	▸ 3.7 git push	
	▸ 3.8 git pull	
	▸ 3.9 Daily Workflow	
Module 4	Branching & Merging	14
	▸ 4.1 Why Branching Matters	
	▸ 4.2 Branch Commands	

- 4.3 git merge
- 4.4 Merge Conflicts — Fix Step by Step
- 4.5 Branch Naming Conventions

Module 5	GitHub Collaboration	16

- 5.1 Fork vs Clone
- 5.2 Pull Requests
- 5.3 Code Review
- 5.4 Issues — Bug & Task Tracking
- 5.5 Adding Collaborators

Module 6	GitHub Features	18

- 6.1 GitHub Pages — Free Hosting
- 6.2 How to Host an HTML File (Step by Step)
- 6.3 GitHub Pages Settings Deep Dive
- 6.4 Custom Domain Setup
- 6.5 GitHub Actions — CI/CD
- 6.6 GitHub Copilot
- 6.7 Semantic Versioning

Module 7	Intermediate Topics	25

- .gitignore
- Staging Area Deep Dive
- Undoing Changes
- Git Stash

Module 8	Advanced Industry-Level Topics	27

- Rebase vs Merge
- Squashing Commits
- Cherry-Pick
- Workflow Strategies
- CI/CD Pipelines

Module 9	3 Hands-On Projects	29

- Project 1: Upload HTML (Beginner)
- Project 2: Team Collaboration (Intermediate)

- Project 3: CI/CD Deployment (Advanced)

Module 10	Interview Preparation	31

- 8 Conceptual Q&A
- Viva Quick Answer Table
- Scenario-Based Questions

Module 11	Cheat Sheet & Error Reference	34

- 60+ Command Quick Reference
- Common Errors & Fixes
- What NOT to Do ⛔
- Pro Tips from Real Developers ⭐

Module 12	Resume Tips & Career Strategy	37

- GitHub Profile Checklist
- Resume Project Listing
- What to Say vs Not Say
- Interview Differentiation Tips

60+ Commands · 3 Projects · 8 Interview Q&A · Complete Cheat Sheet · Pro Tips · Career Guide

MODULE 1: Fundamentals — Git & GitHub

1.1 What is Version Control?

Real-Life Analogy: Imagine you're writing your final year project report. You save it as report.docx, report_final.docx, report_ACTUAL_final.docx... Sound familiar? A Version Control System (VCS) does this automatically and much better.

Three Types of VCS:

- LOCAL VCS — Saves changes only on your computer
- CENTRALIZED VCS — One server, everyone connects (old style: SVN)
- DISTRIBUTED VCS — Everyone has a full copy of history (Git — modern standard)

💡 KEY INSIGHT	A VCS records every change to your files, who made it and when, and lets you restore any previous version instantly. It also lets multiple people work on the same file simultaneously without overwriting each other.

1.2 What is Git?

Git is a free, open-source, distributed version control system created by Linus Torvalds in 2005 — the same person who created the Linux kernel.

- Git runs locally on your computer
- It tracks changes to files over time
- It works completely offline
- It is the industry standard — used by Google, Microsoft, Meta, Amazon, and every major tech company

🕐 ANALOGY	Think of Git as a time machine for your code. Every 'save point' (commit) is a snapshot you can return to at any time.

1.3 What is GitHub?

GitHub is a cloud-based platform that hosts Git repositories online. Created in 2008, acquired by Microsoft in 2018 for $7.5 billion.

GitHub adds on top of Git:

- Cloud storage for your code — accessible from anywhere
- Collaboration tools — pull requests, code reviews, discussions
- Project management — issues, milestones, project boards
- Deployment — GitHub Pages, GitHub Actions CI/CD

- Discovery — browse millions of open-source projects
- Portfolio — your public profile showcases your work to recruiters

1.4 Git vs GitHub — The Critical Difference

Feature	Git	GitHub
What it is	Software (local tool)	Website (cloud platform)
Where it runs	Your local computer	Cloud / Internet
Works offline?	✅ Yes	❌ No
Created by	Linus Torvalds (2005)	Tom Preston-Werner (2008)
Cost	Always free	Free (with paid tiers)
Purpose	Track code changes locally	Host + collaborate on code
Analogy	The actual photo album	Instagram (sharing app)

⚠ REMEMBER	Git can work WITHOUT GitHub. GitHub CANNOT work without Git. Git is the engine; GitHub is the dashboard.

1.5 Core Vocabulary — Must-Know Terms

Term	Simple Definition
Repository (Repo)	A folder that Git is tracking. Contains all project files + complete history of every change.
Commit	A 'save point' — a snapshot of your project at a specific moment. Has a unique ID, message, author, and timestamp.
Branch	A parallel version of your project. Default is 'main'. Lets you develop features without touching stable code.
Clone	Copying a remote repository (from GitHub) to your local computer.
Fork	Creating your own copy of someone else's repository on GitHub. Used for contributing to open-source.
Pull Request (PR)	A formal request to merge your branch's

	changes into another branch. The main collaboration mechanism.
Push	Upload your local commits to GitHub (remote repository).
Pull	Download the latest changes from GitHub to your local computer.
Merge	Combining changes from one branch into another.
HEAD	A pointer to the current commit — where you currently are in history.

MODULE 2: Setup & Installation

2.1 Install Git

Windows:

1. Go to: git-scm.com/download/win and download the 64-bit installer
2. Run installer — keep all default settings
3. Select 'Git Bash' as default terminal (recommended)
4. Verify: Open Git Bash and run: git --version

macOS:

```
xcode-select --install    # Option 1: Xcode tools
brew install git          # Option 2: Homebrew (recommended)
```

Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install git
git --version
```

2.2 First-Time Git Configuration

After installing, tell Git who you are. This info is attached to every commit you make.

```
# Set your full name
git config --global user.name "Pavi"

# Set email (SAME as your GitHub account email)
git config --global user.email "pavi@example.com"

# Set default branch name to 'main'
git config --global init.defaultBranch main

# Set VS Code as default editor (optional)
git config --global core.editor "code --wait"

# Verify everything is set correctly
git config --list
```

2.3 Connect Git to GitHub

Method A: HTTPS + Personal Access Token

5. GitHub → Settings → Developer Settings → Personal Access Tokens → Tokens (classic)
6. Click 'Generate new token (classic)'
7. Set expiry: 90 days | Check scope: ✅ repo
8. COPY the token immediately — you won't see it again!
9. Use the token as your password when Git asks

```
# Cache credentials so you don't retype every time
git config --global credential.helper store
```

Method B: SSH Keys (Recommended for daily use)

```
# Step 1: Generate SSH key
ssh-keygen -t ed25519 -C "pavi@example.com"
# Press Enter 3 times to accept defaults

# Step 2: Copy public key (macOS)
cat ~/.ssh/id_ed25519.pub | pbcopy

# Step 3: Go to GitHub → Settings → SSH Keys → New SSH Key → Paste

# Step 4: Test the connection
ssh -T git@github.com
# Expected: Hi username! You've successfully authenticated...
```

MODULE 3: Basic Commands

3.1 git init — Start a Repository

```
mkdir my-first-project
cd my-first-project
git init

# Output:
# Initialized empty Git repository in /home/user/my-first-project/.git/
```

📁 NOTE	A hidden .git/ folder is created — this is where Git stores ALL history. Never delete or modify it manually.

3.2 git clone — Copy a Repository

```
# Clone using HTTPS
git clone https://github.com/username/repo-name.git

# Clone using SSH (if SSH is configured)
git clone git@github.com:username/repo-name.git

# Clone into a custom folder name
git clone https://github.com/username/repo.git my-folder

# Shallow clone — only latest commit (faster for large repos)
git clone --depth 1 https://github.com/username/repo.git
```

3.3 git status — What's Happening?

Your most-used command. Run this constantly. It shows the current state of your working directory.

```
git status

# Example output:
On branch main

Changes not staged for commit:   ← Modified tracked files
      modified:   index.html

Untracked files:                 ← New files Git doesn't know
      style.css
```

Status	Meaning
Untracked	New file Git has never seen before
Modified	File Git knows about, but it has changed
Staged	File ready to be included in next commit
Committed	File safely saved in Git history

3.4 git add — Stage Your Changes

```
git add index.html          # Stage a specific file
git add index.html style.css  # Stage multiple files
git add .                   # Stage ALL changes in current folder
git add -u                  # Stage all tracked modified files
git add -p index.html       # Stage parts of a file (interactive)
```

🧳 ANALOGY	Staging is like packing a suitcase. You choose exactly which clothes (files) to pack (stage) before zipping it up (committing). You don't have to pack everything — just what you want for this trip.

3.5 git commit — Save a Snapshot

```
git commit -m 'Add navigation bar to index.html'  # Correct way

git commit -am 'Fix typo in about page'         # Stage + commit (tracked only)

git commit --amend -m 'Better message'          # Edit last commit message
```

Writing Good Commit Messages:

Bad Examples (avoid!)	Good Examples (use these!)
❌ fix	❌ changes
❌ asdfgh	❌ FINAL
✅ Add user login form with email validation	✅ Fix navigation bar overflow on mobile
✅ Remove unused CSS variables from style.css	✅ Update README with installation guide

Conventional Commits Standard (Used by Companies):

```
feat:     New feature    → git commit -m 'feat: add search functionality'
fix:      Bug fix        → git commit -m 'fix: resolve login form crash'
docs:     Documentation  → git commit -m 'docs: update API reference'
style:    Formatting     → git commit -m 'style: format with prettier'
refactor: Code cleanup   → git commit -m 'refactor: extract auth module'
test:     Tests          → git commit -m 'test: add login unit tests'
chore:    Maintenance    → git commit -m 'chore: update dependencies'
```

3.6 git log — View History

```
git log                      # Full detailed log
git log --oneline            # Compact one-line view (most useful!)
git log --oneline -5         # Last 5 commits only
git log --oneline --graph --all  # Visual branch graph (beautiful!)
git log --author='Pavi'      # Filter by author
git log --oneline index.html     # History of one specific file
```

3.7 git push — Upload to GitHub

```
git push origin main      # Push to remote 'origin', branch 'main'
git push -u origin main   # First push — sets upstream tracking
git push                  # After -u is set, just this!
git push --all origin     # Push ALL branches
# ⚠     git push --force     # DANGEROUS — only when you know why
```

3.8 git pull — Download from GitHub

```
git pull origin main  # Download and merge from remote
git pull              # If upstream is set

# Or separately:
git fetch origin      # Download only (don't merge yet)
git merge origin/main # Then merge manually
```

Command	Behaviour

git fetch	Downloads changes, does NOT apply. Safe anytime.
git pull	Downloads AND immediately merges. Can cause conflicts.

💡 GOLDEN RULE	ALWAYS run 'git pull' before starting work each day and before 'git push'. This prevents 90% of merge conflicts.

3.9 The Complete Daily Workflow

```
# === START OF WORK SESSION ===
git pull                    # Get latest from teammates

# ... make your changes to files ...

git status                  # See what changed
git add .                   # Stage all changes
git commit -m 'feat: add X feature'  # Save snapshot
git push                    # Upload to GitHub

# === REPEAT for every logical unit of work ===
```

MODULE 4: Branching & Merging

4.1 Why Branching Matters

Branching lets you create a parallel copy of your code to work on features or fixes — without touching the stable main codebase.

```
# WITHOUT branching (bad):
main: A → B → C → [broken code!] → D → E
                ↑ Everyone's work is now broken!

# WITH branching (professional):
main:    A → B → C ──────────────────────→ Merge → E
             ↓              ↑
feature:       C → F → G → H (tested) ↗
```

⚠ RULE	NEVER commit directly to 'main'. Always work on a feature branch. This is a universal industry standard.

4.2 Branch Commands

```
git branch                  # List all local branches
git branch -a               # List all (including remote) branches
git branch feature-login    # Create a new branch
git switch feature-login    # Switch to branch (modern)
git switch -c feature-login # Create AND switch (modern — use this!)
git checkout -b feature-login # Create AND switch (older way)
git branch -d feature-login # Delete merged branch
git branch -D feature-login # Force delete (even if unmerged)
git branch -m new-name      # Rename current branch
```

4.3 git merge — Combine Branches

```
# Step 1: Switch to the branch you merge INTO (usually main)
git switch main

# Step 2: Merge your feature branch
git merge feature-login

# Output:
# Updating a3f8c2d..f1e2d3c
# Fast-forward
```

```
# login.html | 45 ++++++++++++++++
```

4.4 Merge Conflicts — Understanding & Fixing

A merge conflict occurs when two branches change the SAME LINE of the SAME FILE. Git marks the conflict and asks you to decide.

What a Conflict Looks Like in the File:

```
<<<<<<< HEAD              ← Your current branch version
<h1>Welcome</h1>
=======
<h1>Hello World</h1>        ← The branch being merged in
>>>>>>> feature-login
```

Step-by-Step Resolution:

10. Run: git merge feature-login → CONFLICT error appears
11. Run: git status → see which files have conflicts
12. Open the file, MANUALLY edit it — delete markers, keep what you want
13. Run: git add index.html
14. Run: git commit -m 'merge: resolve conflict in index.html'

💡 TIP	VS Code has a built-in conflict resolver showing 'Accept Current / Accept Incoming / Accept Both'. This makes resolution much easier than editing raw markers.

4.5 Professional Branch Naming Conventions

```
feature/user-login        ← New feature
fix/navigation-overflow   ← Bug fix
hotfix/payment-crash      ← Urgent production fix
docs/api-guide            ← Documentation update
test/login-unit-tests     ← Adding tests
refactor/auth-module      ← Code restructure
```

MODULE 5: GitHub Collaboration

5.1 Fork vs Clone

Feature	Fork	Clone
Location	On GitHub (creates copy under YOUR account)	On your computer
Use case	Contributing to someone else's project	Working on a project you have access to
Can push directly?	Only to your fork	Yes (if you have permission)
How to do it	Button on GitHub website	git clone <url>

5.2 Pull Requests — The Heart of GitHub Collaboration

A Pull Request (PR) is a formal request to merge your branch's changes into another branch. It includes the code diff, a description, and a comment thread for review.

Creating a PR:

15. Push your feature branch to GitHub
16. GitHub shows: 'Compare & pull request' banner → Click it
17. Fill in: Title, Description, and assign Reviewers
18. Click 'Create Pull Request'

Professional PR Description Template:

```
## What does this PR do?
Adds JWT-based user authentication.

## Changes Made
- Added auth.js with login/logout functions
- Added /login route in app.js

## How to Test
1. Run npm start
2. Navigate to /login
3. Use: admin@test.com / password123

## Checklist
- [x] Code works locally
```

```
- [x] README updated
- [ ] Unit tests added
```

5.3 Issues — Tracking Bugs & Tasks

Issue Type	Example
Bug Report	Login page crashes on mobile Safari
Feature Request	Add dark mode toggle to settings
Task	Write API documentation for v2
Question	How do I set up the dev environment?

Pro Tip: Use 'closes #12' in your commit message to automatically close Issue #12 when the commit is merged to main.

MODULE 6: GitHub Features

6.1 GitHub Pages — Free Web Hosting Overview

GitHub Pages is a FREE static site hosting service built into every GitHub repository. It turns your code directly into a live, publicly accessible website — no server, no domain purchase, no cost.

Your URL format:

```
Personal/Portfolio site:  https://yourusername.github.io
Project site:             https://yourusername.github.io/repo-name

Example:
Username = pavi2024, Repo = my-portfolio
→ Live URL: https://pavi2024.github.io/my-portfolio
```

✅ Can Host	❌ Cannot Host
✅ HTML/CSS/JS websites	❌ Backend code (Node.js, Python, PHP)
✅ React/Vue apps (after build)	❌ Apps that need a database
✅ Jekyll & Hugo blogs	❌ Real-time apps (chat, live data)
✅ Documentation sites	❌ Apps with user login/sessions (server-side)
✅ Portfolio sites	❌ APIs or REST endpoints

6.2 How to Host an HTML File — Complete Step-by-Step

This is the full process from creating your HTML file to seeing it live on the internet.

PHASE 1 — Prepare Your HTML Project

19. Create a folder for your project on your computer
20. Inside it, create index.html — this MUST be named index.html (it is the homepage)
21. Optionally add style.css, script.js, images/ folder alongside it

Sample index.html to use:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My GitHub Pages Site</title>
  <style>
    body { font-family: Arial, sans-serif; text-align: center; padding: 60px; }
    h1   { color: #065A82; }
    p    { color: #555; font-size: 18px; }
  </style>
</head>
<body>
  <h1>🚀 Hello from GitHub Pages!</h1>
  <p>This site is hosted for FREE using GitHub Pages.</p>
  <p>Uploaded by: <strong>Pavi</strong></p>
</body>
</html>
```

PHASE 2 — Create GitHub Repository

22. Go to github.com and sign in to your account
23. Click the + icon (top-right) → select 'New repository'
24. Fill in the form:
    - Repository name: my-portfolio (use hyphens, no spaces)
    - Description: My personal portfolio site (optional)
    - Visibility: Public (REQUIRED for free GitHub Pages)
    - ✅ CHECK 'Add a README file' — this initializes the repo with a main branch
25. Click 'Create repository'

⚠ IMPORTANT	Your repository MUST be set to Public to use GitHub Pages on the free plan. Private repos require a paid GitHub plan.

PHASE 3 — Upload Your HTML Files

METHOD A: Upload via GitHub Website (Beginner — no Git commands needed):

26. Open your newly created repository on GitHub
27. Click 'Add file' button → select 'Upload files'
28. Drag and drop your index.html (and style.css, images/ etc.) into the upload area
29. Scroll down to 'Commit changes' section
30. Write a commit message: Add index.html homepage
31. Ensure 'Commit directly to the main branch' is selected
32. Click 'Commit changes' — your files now appear in the repo!

METHOD B: Upload via Git Commands (Recommended):

```
# Step 1: Open terminal in your project folder
cd /path/to/my-portfolio

# Step 2: Initialize Git
git init

# Step 3: Connect to your GitHub repository
git remote add origin https://github.com/yourusername/my-portfolio.git

# Step 4: Stage all files
git add .

# Step 5: Commit
git commit -m "feat: add portfolio homepage"

# Step 6: Push to GitHub
git branch -M main
git push -u origin main
```

PHASE 4 — Enable GitHub Pages

33. On your GitHub repository page, click the ⚙ Settings tab (top menu)
34. In the LEFT sidebar, scroll down and click 'Pages' under the 'Code and automation' section
35. Under 'Build and deployment' → 'Source', click the dropdown that says 'None'
36. Select 'Deploy from a branch'
37. Under 'Branch', click the branch dropdown and select 'main'
38. In the folder dropdown next to it, keep it as '/(root)'
39. Click the 'Save' button

PHASE 5 — View Your Live Website

40. Stay on the Settings → Pages screen
41. Wait approximately 30 seconds to 2 minutes
42. Refresh the page — a blue/green banner appears:
    - 'Your site is live at https://yourusername.github.io/my-portfolio'
43. Click the 'Visit site' button or open the URL in a new tab
44. Your HTML file is now a LIVE website!

🚀 LIVE	Your site URL is permanent and shareable. Add it to your resume, LinkedIn, and share with professors and recruiters. Every time you push new code to

	main, the site automatically updates within 1-2 minutes.

6.3 GitHub Pages Settings — Deep Dive

Understanding every option on the GitHub Pages settings screen:

Setting	Explanation
Source	The deployment method. 'Deploy from a branch' = simplest. 'GitHub Actions' = advanced CI/CD control.
Branch (dropdown)	Which branch to serve. Always use 'main' unless you have a specific gh-pages branch setup.
Folder (dropdown)	'/(root)' serves files from the root of the branch. '/docs' serves only from a /docs subfolder — useful for project documentation sites.
Custom domain	Enter a domain you own (e.g., mysite.com) to replace the .github.io URL. Requires DNS configuration at your domain registrar.
Enforce HTTPS	Forces all visitors to use HTTPS (secure connection). Always keep this ON for security.
Visibility	Public (anyone can see). If your repo is private, the site is also private (requires GitHub Pro/Teams).
Your site is live at:	The generated public URL. This appears after first deployment is complete.

Common GitHub Pages Troubleshooting:

Problem	Solution
Site shows 404 error	File is NOT named index.html exactly (check capitalization)
CSS not loading	CSS file path in HTML is wrong. Use relative paths: href='style.css' not href='/style.css'
Changes not showing	Wait 2-3 min and hard-refresh browser

	(Ctrl+Shift+R). Deployment takes time.
404 on Settings → Pages	Repo may be private. Switch to Public in Settings → General → Danger Zone
Images broken on live site	Image paths must be lowercase, no spaces. Use: src='images/photo.jpg'
Custom domain shows wrong content	DNS propagation takes 24-48 hours. Be patient after adding CNAME record.

6.4 Custom Domain Setup (Optional)

Replace your .github.io URL with your own domain (e.g., pavi.dev or pavi.in).

Step 1: Buy a Domain

- Namecheap, GoDaddy, Google Domains, or Hostinger
- .in domains cost ~₹600-800/year. .com costs ~₹800-1200/year

Step 2: Add CNAME File to Your Repo

```
# Create a file named CNAME (no extension) in your repo root
# File contents — just your domain, nothing else:
mysite.com
```

Step 3: Configure DNS at Your Domain Registrar

```
Add these A records (pointing to GitHub's servers):
185.199.108.153
185.199.109.153
185.199.110.153
185.199.111.153

And a CNAME record:
www  →  yourusername.github.io
```

Step 4: Set Custom Domain in GitHub Pages Settings

45. Settings → Pages → Custom domain → Enter your domain → Save
46. Wait 24-48 hours for DNS propagation
47. Check 'Enforce HTTPS' after DNS is verified

💡 TIP	For a student portfolio, a custom domain like yourname.dev or yourname.in looks extremely professional on your resume. It costs less than a movie ticket per month.

6.5 GitHub Actions — CI/CD Basics

Automated workflows that run on GitHub servers when specific events happen (push, PR, schedule).

Example: Auto-run tests on every push (.github/workflows/ci.yml):

```yaml
name: CI Pipeline

on:                 # When to run
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm install
      - run: npm test
      - run: npm run build
```

💼 CAREER TIP	Companies run CI/CD on every PR. Knowing GitHub Actions is a highly valued skill. Mention it in your resume and interviews!

6.6 GitHub Copilot — AI Coding Assistant

- Auto-completes entire functions based on comments
- Suggests code as you type in VS Code
- Generates unit tests and explains complex code
- FREE for students: Apply at education.github.com/pack

⚠ IMPORTANT	Copilot makes suggestions — YOU must review them. It sometimes suggests incorrect or insecure code. Never accept blindly.

6.7 Semantic Versioning (SemVer)

```
v MAJOR . MINOR . PATCH
v 1   . 4   . 2

MAJOR: Breaking changes (old code won't work)
MINOR: New features (backward compatible)
PATCH: Bug fixes (backward compatible)

Examples:
v1.0.0 → v1.0.1  Bug fix
v1.0.1 → v1.1.0  New feature added
v1.1.0 → v2.0.0  Major rewrite — breaking changes
```

MODULE 7: Intermediate Topics

7.1 .gitignore — What Not to Track

A .gitignore file tells Git to completely ignore certain files and folders. CRITICAL for security.

```
# Dependencies — never commit these
node_modules/
vendor/

# SECURITY: Environment variables with secrets
.env
.env.local
.env.production

# Build outputs
dist/
build/

# IDE files
.vscode/
.idea/

# OS files
.DS_Store        # Mac
Thumbs.db        # Windows

# Python
__pycache__/
*.pyc
venv/
```

⚠ SECURITY WARNING	Committing a .env file with API keys or passwords to a PUBLIC repository is a serious security incident. Companies have incurred financial damages from this mistake. Always check .gitignore before committing!

7.2 Undoing Changes — The Safety Net

Situation	Command
Changed file, not staged	git restore <file>
File staged, not committed	git restore --staged <file>
Just committed, not pushed	git reset --soft HEAD~1
Committed and pushed (shared!)	git revert <SHA> ← Always use this

```
git restore index.html        # Discard changes in file
git restore --staged index.html # Unstage a file
git reset --soft HEAD~1       # Undo commit, keep changes staged
git reset --mixed HEAD~1      # Undo commit, keep files (not staged)
git reset --hard HEAD~1       # ⚠ DANGER: Undo commit + delete changes
git revert HEAD               # Safe undo — creates reverting commit
```

7.3 Git Stash — Temporarily Shelve Work

```
git stash                  # Stash current changes
git stash save "WIP: login"  # Stash with a label
git stash list             # See all stashes
git stash pop              # Apply latest stash + remove it
git stash apply            # Apply latest stash (keep in list)
git stash drop stash@{0}   # Delete a stash
git stash clear            # Delete ALL stashes
```

💡 REAL-WORLD USE	You're in the middle of a feature when your manager asks for an urgent bug fix. git stash your work, fix the bug, git push, then git stash pop to resume your feature.

MODULE 8: Advanced Industry-Level Topics

8.1 Rebase vs Merge

Feature	Merge	Rebase
Creates merge commit?	✅ Yes	❌ No (rewrites history)
History shape	Non-linear (branches visible)	Linear (looks sequential)
Safe for shared branches?	✅ Yes	⚠ No — never rebase shared branches
Best for	Team feature branches	Personal branch updates

```
# Merge (safe, preserves history)
git switch main && git merge feature/login

# Rebase (cleaner linear history — for personal branches only!)
git switch feature/login && git rebase main

# Interactive rebase — squash/reorder/edit last 3 commits
git rebase -i HEAD~3
```

⚠ GOLDEN RULE	NEVER rebase commits that have been pushed to a shared/public branch. It rewrites history and causes serious problems for everyone on the team.

8.2 Squashing Commits — Clean History Before Merge

Squashing combines multiple small commits into one meaningful commit before merging a PR.

```
# Squash last 4 commits into one
git rebase -i HEAD~4

# In the editor that opens, change 'pick' to 'squash':
pick a3f8c2d Add login page
squash b4e9d1c Fix typo
squash c5f1e2a Add validation
squash d6g2f3b More fixes

# Git asks for new combined commit message:
feat: add complete user login with validation
```

```
# Result: 4 messy commits → 1 clean commit
```

8.3 Cherry-Pick — Grab a Specific Commit

Apply a specific commit from one branch to another, without merging the entire branch.

```
# Find the commit you want
git log --oneline feature/notifications
# b4e9d1c Fix notification styling  ← I want ONLY this

# Apply it to current branch
git cherry-pick b4e9d1c
```

💡 USE CASE	A critical bug fix exists in a feature branch that isn't ready to merge. Cherry-pick just the bug fix commit to main — you get the fix without the incomplete feature.

8.4 Workflow Strategies — Industry Standards

Strategy	Description
Git Flow	Separate main + develop + feature/release/hotfix branches. For products with scheduled releases.
Feature Branch	Branch per feature off main, PR back to main. Most common — used by most startups and mid-size teams.
Trunk-Based	Everyone commits to main multiple times daily. Requires excellent CI/CD. Used by Google, Facebook.
Forking Workflow	Each contributor forks the repo and PRs to original. Used in open-source projects.

MODULE 9: Hands-On Projects

🟢 Project 1 (Beginner): Upload HTML Project

```
# Complete workflow from scratch
mkdir portfolio-website && cd portfolio-website
git init
echo ".DS_Store\nThumbs.db" > .gitignore

# Create index.html (your HTML file goes here)

git add .
git commit -m "feat: initial portfolio website"

# Create repo on GitHub (github.com → + → New repository)
# Name: portfolio-website | Public | NO README (you have files)

git remote add origin https://github.com/YOUR_USERNAME/portfolio-website.git
git branch -M main
git push -u origin main
```

Project Checklist:

- Repository created on GitHub ✅
- HTML file uploaded and visible on GitHub ✅
- .gitignore present ✅
- Meaningful commit message used ✅

🟡 Project 2 (Intermediate): Team Collaboration

Goal: Practice branching, PRs, and resolving a merge conflict with a partner.

48. Person A creates repo and adds Person B as collaborator
49. Person A uploads index.html with a navigation bar, commits to main
50. Person B clones the repo, creates branch feature/add-content
51. Person B adds content to index.html, commits and pushes branch
52. Person B creates Pull Request on GitHub
53. Person A reviews, comments, approves, and merges the PR
54. Both now edit the SAME LINE in index.html on different branches
55. Trigger a merge conflict intentionally — then practice resolving it!

🔴 Project 3 (Advanced): CI/CD with GitHub Actions

Goal: Set up automatic deployment via GitHub Actions whenever you push to main.
Create .github/workflows/deploy.yml:

```yaml
name: Auto Deploy
on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      contents: write
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to GitHub Pages
        uses: peaceiris/actions-gh-pages@v3
        with:
          github_token: ${{ secrets.GITHUB_TOKEN }}
          publish_dir: ./
```

🏆 ACHIEVEMENT	A CI/CD pipeline on your project immediately differentiates you from other candidates. Mention it explicitly in interviews: 'I set up GitHub Actions to auto-deploy on every push to main.'

MODULE 10: Interview Preparation

10.1 Conceptual Questions & Answers

Q1: What is the difference between git merge and git rebase?

Both integrate changes from one branch into another. git merge creates a new 'merge commit' that preserves the complete history — you can see when branches diverged and rejoined. git rebase moves (replays) your commits on top of the target branch, creating a linear history as if the work happened sequentially. Merge is safer for shared public branches; rebase creates cleaner history for personal feature branches. Golden rule: never rebase commits pushed to a shared branch.

Q2: What is a merge conflict and how do you resolve it?

A merge conflict occurs when two branches modify the same line(s) of the same file, and Git cannot automatically determine which version to keep. Git marks conflicted sections with <<<<<<, =======, and >>>>>>> markers. Resolution: open the file, manually edit to keep the desired version, delete conflict markers, then git add the file and git commit to complete the merge.

Q3: What is the difference between git pull and git fetch?

git fetch downloads changes from the remote but does NOT apply them — it just updates local knowledge of the remote. git pull is fetch + merge combined — it downloads AND immediately merges into your current branch. fetch is safer when you want to review changes before applying them.

Q4: Explain the Git staging area.

The staging area (or index) is an intermediate zone between your working directory and the repository. When you modify files, changes start in the working directory. git add moves them to the staging area — a preview of your next commit. git commit saves everything in the staging area as a permanent snapshot. This design lets you craft precise commits by selecting exactly which changes to include from multiple modified files.

Q5: What is a detached HEAD state?

HEAD normally points to the tip of a branch. Detached HEAD occurs when you checkout a specific commit SHA directly instead of a branch name. In this state, new commits are not on any branch and will be lost when you switch. Fix: git switch main to return to a branch, or git switch -c new-branch to save your detached work.

Q6: Difference between git reset and git revert?

git reset moves the branch pointer backward, effectively deleting commits from history — it is destructive and should only be used on unpushed local commits. git revert creates a NEW commit that exactly reverses a previous commit's changes — history is preserved, making it safe for shared branches. On a team project, ALWAYS use git revert for pushed commits.

Q7: What is .gitignore and why is it critical?

.gitignore tells Git which files to exclude from tracking. It is critical for: keeping secrets (API keys, passwords in .env files) out of public repos; avoiding committing large dependency folders (node_modules/); excluding build artifacts. Accidentally committing .env to a public repo is a serious security incident — companies have suffered financial damages because of this.

Q8: What is cherry-pick and when do you use it?

Cherry-pick applies a specific commit from one branch to another without merging the whole branch. Use case: a critical bug fix is on a feature branch that isn't ready to merge to main yet — cherry-pick just the bug fix commit to main.

10.2 Quick Viva Q&A Table

Question	Answer
What does HEAD mean?	Pointer to the current commit (where you are in history)
What is 'origin'?	Default alias for your remote repository URL
What is a shallow clone?	git clone --depth 1 — only latest commit, not full history
Fork vs Branch?	Fork = copy on GitHub; Branch = parallel line in same repo
What does git stash do?	Temporarily saves uncommitted changes to shelve them
What is git bisect?	Binary search to find which commit introduced a bug
What is git blame?	Shows who last modified each line of a file
What is git reflog?	Log of ALL HEAD changes — your safety net for lost commits
What is --amend?	Modifies the most recent commit (message or content)
What is a bare repository?	A repo with no working directory — used on servers

10.3 Scenario-Based Questions

Scenario 1: You pushed an API key to a public GitHub repo. What do you do?

56. IMMEDIATELY revoke/rotate the exposed key in the service dashboard
57. Remove it from code, commit, and push the fix
58. Use git filter-repo to remove it from ALL commit history
59. Force push the cleaned history
60. Add the file to .gitignore so it can never be committed again
61. Assume compromised — anyone may have already copied it

Scenario 2: You committed to main instead of your feature branch.

```
# If NOT pushed yet:
git reset --soft HEAD~1          # Undo commit, keep changes
git switch -c feature/my-feature  # Create correct branch
git commit -m "feat: correct branch"

# If already pushed to main:
git revert HEAD                  # Safer — doesn't rewrite history
```

MODULE 11: Cheat Sheet

11.1 Quick Command Reference

```
# ═══════ SETUP ═══════
git config --global user.name "Name"
git config --global user.email "email@example.com"
git config --list

# ═══════ STARTING ═══════
git init                # Initialize repo
git clone <url>         # Clone remote repo

# ═══════ DAILY WORKFLOW ═══════
git status              # Check state (use constantly!)
git add <file>          # Stage specific file
git add .               # Stage all changes
git commit -m "message"    # Commit staged changes
git push                # Push to remote
git pull                # Pull from remote

# ═══════ BRANCHING ═══════
git branch              # List branches
git switch -c <name>       # Create + switch to branch
git merge <branch>         # Merge branch into current
git branch -d <name>       # Delete merged branch

# ═══════ HISTORY ═══════
git log --oneline --graph --all  # Visual branch graph
git diff --staged              # See staged changes
git blame <file>               # Who changed each line

# ═══════ UNDO ═══════
git restore <file>         # Discard working changes
git restore --staged <file># Unstage file
git reset --soft HEAD~1    # Undo commit, keep staged
git revert <SHA>           # Safe undo (shared branches)

# ═══════ ADVANCED ═══════
git stash               # Shelve changes
git stash pop           # Apply + remove stash
git cherry-pick <SHA>      # Apply specific commit
git rebase -i HEAD~3       # Interactive rebase
git reflog              # Recovery log — LIFESAVER!
```

11.2 Common Errors & Fixes

Error	Cause	Fix
fatal: not a git repository	Not inside a git repo	cd to correct folder or git init
error: failed to push some refs	Remote has commits you don't have	git pull first, then push
CONFLICT: Merge conflict in X	Two branches changed same line	Edit file, git add, git commit
Please tell me who you are	Git not configured	git config --global user.name/email
Permission denied (publickey)	SSH key not on GitHub	Add SSH key in GitHub Settings
fatal: remote origin already exists	Remote already set	git remote set-url origin <new-url>
detached HEAD state	Checked out a commit SHA directly	git switch main
refusing to merge unrelated histories	Repos have no common history	git pull origin main --allow-unrelated-histories

11.3 What NOT to Do ⛔

- NEVER commit .env files — API keys, passwords, secrets belong there
- NEVER force push to shared/main branches (git push --force)
- NEVER rebase commits that have been pushed to shared branches
- NEVER commit node_modules/ or build/ folders
- NEVER write commit messages like 'fix', 'changes', 'update', 'asdf'
- NEVER leave merge conflict markers (<<<<<<) in committed code
- NEVER commit directly to main — always use feature branches
- NEVER merge a PR that fails CI tests

11.4 Pro Tips from Real Developers ⭐

- Commit often, push regularly — small commits are easier to review and revert
- git reflog is your safety net — if you think you lost work, run it first
- Before risky operations, create a backup branch: git branch backup/pre-rebase
- Use 'git switch -' to go back to the previous branch (like 'cd -' in terminal)
- Press '.' on any GitHub repo to open it in VS Code in the browser (instant editor!)
- Pin your 6 best repos on your GitHub profile — recruiters see these first
- git commit --amend fixes the last commit — only use BEFORE pushing!
- 'git log --oneline --graph --all' is the most useful command for understanding repo state

MODULE 12: Resume Tips & Career Strategy

12.1 GitHub Profile Checklist

- Professional profile photo (not cartoon)
- Clear bio: 'ISE Student | Full Stack Dev | Python & JavaScript'
- Location, portfolio link, university filled in
- 6 pinned repositories — your best projects
- Active contribution graph — commit regularly!
- Professional README.md for every pinned project

💡 SPECIAL TIP	Create a repo named 'your-username/your-username' — its README.md appears directly on your GitHub profile page. Use this as your developer business card!

12.2 How to List Projects on Your Resume

```
PROJECTS
──────────────────────────────────────────────────────────────────

Real-Time Chat Application                Jan 2026
• Built using Node.js, Express.js, and Socket.io
• Supports 50+ concurrent users with room-based messaging
• Deployed on Render with MongoDB Atlas database
• GitHub: github.com/pavi/realtime-chat
• Live Demo: pavi.github.io/chat-demo

Skills Section:
Version Control: Git, GitHub (branching, PRs, code review,
                 GitHub Actions CI/CD)
```

12.3 What to Say vs What NOT to Say

Avoid (Weak)	Use Instead (Strong)
❌ 'Familiar with GitHub'	✅ 'Collaborated using Git/GitHub: feature branches, PRs, code review'
❌ 'Used GitHub to store code'	✅ 'Set up GitHub Actions CI/CD pipeline for automated testing and deployment'
❌ 'Pushed files to GitHub'	✅ 'Managed a 3-person project using feature branch workflow and pull requests'

12.4 How to Stand Out in Interviews

62. Before interview: clean up top 3 repos — good README, working code, live demo
63. Know your own code: be ready to explain technical decisions you made
64. Proactively say: 'You can view my projects at github.com/pavi'
65. Mention CI/CD explicitly: 'I set up GitHub Actions to auto-deploy on every push to main'
66. Show contribution graph activity — consistent commits demonstrate discipline

Final Words 🎓

Learning Git and GitHub is not about memorizing commands. It's about building a workflow habit. The commands become second nature within weeks of daily practice.

The developers who stand out in internships are not always the best coders — they're the ones who commit clearly, communicate through PRs, keep their code organized, and never break the main branch. Start today. Create your first repo. Make your first commit. The rest will follow.

GitHub Complete Mastery Guide · ISE Student Edition 2026
60+ Commands · 3 Projects · 8 Interview Q&A · Complete Cheat Sheet